Task-1 : Prepare a test strategy document based on the domain, using a chain-of-thought approach

structured the way a real QA team would expect.

✅ **Functional Testcases – Same Bank Fund Transfer**

TC-ID	Priority	Test Description	Precondition	Test Steps	Test Data	Expected Result	Actual Result
TC001	P0	Verify successful fund transfer with valid details	User logged in, sufficient balance available	1. Login 2. Navigate to Fund Transfer 3. Enter valid beneficiary account 4. Enter valid amount 5. Submit	Acc1: 111111 Acc2: 222222 Amount: 5000	Transfer should complete successfully and balance updated	TBD
TC002	P0	Verify transfer fails when insufficient balance	User logged in, insufficient balance	1. Login 2. Enter transfer details 3. Submit	Balance: 1000 Transfer: 5000	System should show "Insufficient Balance" error	TBD
TC003	P0	Verify transfer to invalid account number	User logged in	1. Login 2. Enter invalid account number 3. Submit	Acc2: 999999 (invalid)	System should show "Invalid Account Number"	TBD
TC004	P1	Verify transfer with minimum allowed amount	User logged in	1. Login 2. Enter minimum transfer amount 3. Submit	Amount: 1	Transfer should be successful	TBD
TC005	P1	Verify transfer exceeding daily limit	User logged in	1. Login 2. Enter amount > daily limit 3. Submit	Amount: 1000000	System should block transfer with limit message	TBD

TC-ID	Priority	Test Description	Precondition	Test Steps	Test Data	Expected Result	Actual Result
TC006	P1	Verify transfer with empty fields	User logged in	1. Login 2. Leave fields blank 3. Click submit	NA	System should show validation errors	TBD
TC007	P2	Verify transfer cancellation before submission	User logged in	1. Login 2. Enter details 3. Click cancel	Valid data	Transfer should not be processed	TBD
TC008	P2	Verify transfer with special characters in account number	User logged in	1. Login 2. Enter account with special chars 3. Submit	Acc2: @@@###	System should reject invalid format	TBD
TC009	P0	Verify transfer between same account (self-transfer)	User logged in	1. Login 2. Enter same account number 3. Submit	Acc1 = Acc2	System should prevent self-transfer	TBD
TC010	P1	Verify transfer session timeout handling	User logged in, session expired	1. Login 2. Wait for timeout 3. Attempt transfer	Valid data	System should redirect to login or show session expired	TBD

🔍 Team Lead Review Comments (Brutally Honest)

- You covered **core happy path + major negative scenarios**, which is good — but don't get comfortable.
- Missing **real banking complexity**:
  - No **beneficiary addition/validation flow** (huge gap)
  - No **OTP / 2FA validation cases** → unacceptable for banking
  - No **concurrency / duplicate transaction handling**
- Your **P0 tagging is decent**, but borderline:
  - TC009 (self-transfer) is correctly P0 👍
  - TC002, TC003 also correct P0 — good prioritization thinking

- Test steps are **too generic**:
  - Real automation-ready testcases need **element-level clarity**
- No **boundary value testing** beyond minimum — where is:
  - max amount boundary?
  - decimal handling?
- "Actual Result = TBD" is fine for template — but in execution phase, this is where most testers mess up by writing useless comments.

Task-2 : Design the automation framework structure using a chain-of-thought approach

Prompt

Context: As an automation tester, generate the Selenium automation framework

Instruction :

1. Ask me 6 questions for the framework creation
2. Ask the next question after I answered the first question
3. Framework should be an industrial standard

Persona Act as a Test Automation Architect to review the test cases and give review comments

Tone: [STRICTLY] Professional tone in review comments and summary

Let's build this properly—no shortcuts, no vague assumptions.

Question 1 (Foundation – Critical Decision)

What type of framework architecture do you want?

Choose **one** (don't say "mix of everything" unless you can justify it):

1. **Hybrid Framework (Recommended – Industry Standard)**
   → Combines POM + Data-Driven + Utility layers
2. **Page Object Model (POM) only**
   → Cleaner but limited scalability
3. **BDD (Cucumber) with POM**
   → Useful for business-readable tests
4. **Keyword-Driven Framework**
   → Rarely used nowadays, harder to maintain

👉 Don't answer casually. Your choice here impacts:

- Maintainability
- Scalability
- CI/CD integration
- Team collaboration

Answer with one option + reason.
Then I'll move to Question 2.

Hybrid Framework

Good. That's the only sensible choice if you're aiming for real-world scalability.

But just selecting "Hybrid" is meaningless unless you understand what you're combining.

Quick Reality Check

If your "Hybrid" = just POM + Excel → that's **basic**, not industry-grade.

A proper Hybrid Framework should include:

- POM (Page layer)
- Data-driven (external sources)
- Utilities (waits, drivers, configs)
- Reporting
- Logging
- Environment handling
- CI/CD readiness

If you're not planning all of this, you're building a **toy framework**, not an industrial one.

Question 2 (Execution Engine – Backbone)

Which test runner + build tool combination do you want?

Pick one:

1. **TestNG + Maven (Industry Standard – Strongly Recommended)**
2. **JUnit + Maven**
3. **TestNG + Gradle**
4. **Cucumber + TestNG + Maven (BDD Hybrid)**

👉 Be careful:

- If you pick Cucumber without real need → unnecessary complexity
- If you skip build tools → your framework is already broken

Answer with one option + reason.

TestNG + Maven

Correct choice. This is the backbone of most stable enterprise frameworks.

But don't assume this alone makes your framework "industrial."
If you misuse TestNG (no proper suite design, no parallel control, no listeners), it becomes messy fast.

Question 3 (Design Pattern – Maintainability Core)

How do you want to design your Page Layer?

Pick one:

1. **Classic Page Object Model (POM)**
   → One class per page, locators + actions together
2. **Page Factory (with @FindBy)**
   → Cleaner syntax, but often misused and less flexible
3. **POM + Separate Action Layer (Recommended – Advanced)**
   → Page = locators only
   → Actions = reusable methods (click, type, etc.)
4. **POM + Fluent Interface (Advanced)**
   → Method chaining (e.g., loginPage.enterUser().enterPass().clickLogin())

Reality Check

- If you pick only basic POM → your framework will grow messy
- If you over-engineer (Fluent everywhere) → becomes hard to debug
- Industry standard in mature teams = **POM + reusable action/util layer**

👉 **Answer with one option + reason.**
Then we move to Question 4.

POM + Fluent Interface (Advanced)

Alright — but this is where I push back.

Brutal Reality Check

POM + Fluent Interface sounds "advanced", but most teams misuse it.

Problems you'll hit if you're not careful:

- Debugging becomes painful (chained calls hide failures)
- Logs become unclear

- Reusability suffers if chaining is forced everywhere
- New team members struggle to maintain it

👉 Fluent Interface is useful **only for high-level business flows**, not for every method.

What mature teams actually do:

- POM + Utility/Action Layer (core)
- Fluent Interface → used selectively (login, checkout flows)

If you blindly apply Fluent everywhere → you're building a fragile framework.

I'll allow this choice, but only if you apply it **strategically, not blindly**.

Question 4 (Execution Strategy – Scaling & Performance)

How do you want to handle test execution?

Pick one:

1. **Sequential Execution (Basic – Not acceptable for real projects)**
2. **Parallel Execution using TestNG (Recommended)**
3. **Parallel + ThreadLocal WebDriver (Industry Standard)**
4. **Distributed Execution (Grid / Cloud + Parallel)**

Reality Check

- If your framework doesn't support parallel → it's already outdated
- If you use parallel without ThreadLocal → expect random failures
- If you plan CI/CD → parallel is **mandatory**

👉 **Answer with one option + reason.**
Then we move to Question 5.

Parallel + ThreadLocal WebDriver

Good. That's the correct engineering choice.

Reality Check (Important)

Most people *say* ThreadLocal, but implement it wrong:

- Static WebDriver → ❌ breaks parallel
- Shared instances → ❌ flaky tests

- Improper cleanup → ❌ memory leaks

If you don't manage:

- `ThreadLocal<WebDriver>`
- Proper `set()` and `get()`
- `remove()` after execution

→ Your framework will become unstable under load.

Question 5 (Test Data Strategy – Scalability Bottleneck)

How do you want to manage test data?

Pick one:

1. **Hardcoded Data (Unacceptable)**
2. **Excel (Apache POI)**
3. **JSON / YAML (Recommended for modern frameworks)**
4. **Database (Advanced – enterprise-level)**
5. **Combination (Excel + JSON + DB)**

Reality Check

- Excel is overused and slows execution
- JSON is cleaner, version-control friendly
- DB is powerful but overkill unless truly needed
- Hybrid approach is best *only if managed properly*

👉 **Answer with one option + reason.**
Then final Question 6.

Excel

That's a **practical but slightly outdated choice**—acceptable, but not optimal.

Reality Check

Excel works, but comes with baggage:

- Slower read operations (Apache POI overhead)
- Hard to maintain at scale
- Poor version control visibility (compared to JSON)
- Becomes messy with large datasets

👉 Still widely used in enterprise projects, so **acceptable if structured properly**:

- Separate sheets per module
- Avoid excessive columns
- Build a proper utility layer (don't read Excel in test classes directly)

Question 6 (Final – Reporting & Observability)

How do you want to handle reporting and logging?

Pick one:

1. **Extent Reports only**
2. **Log4j only**
3. **Extent Reports + Log4j (Industry Standard – Recommended)**
4. **Allure Reports + Logging (Modern alternative)**

Reality Check

- Reporting without logging → useless for debugging
- Logging without reporting → useless for stakeholders
- If your reports don't include screenshots → poor framework

👉 **Answer with one option + reason.**

After this, I will generate your **complete industrial-grade Selenium framework structure + design + best practices**.

Extent Reports + Log4j

Good. That's the correct combination for both **visibility (reports)** and **debugging (logs)**.

✅ Final Framework Design (Industrial Standard)

🔷 1. Framework Architecture (Hybrid)

Your framework will include:

```
src/main/java
 ├── base
 │    └── BaseTest.java
 │    └── DriverManager.java (ThreadLocal)
 │
```

```
├── pages
│    └── LoginPage.java
│    └── DashboardPage.java
│
├── utils
│    └── ExcelUtil.java
│    └── ConfigReader.java
│    └── WaitUtil.java
│    └── ScreenshotUtil.java
│
├── actions (Fluent Interface - controlled usage)
│    └── LoginActions.java
│
├── listeners
│    └── TestListener.java (Extent Reports)
│
├── reports
│    └── ExtentManager.java
│
├── constants
│    └── FrameworkConstants.java
│
└── enums (optional but recommended)
     └── WaitStrategy.java


src/test/java
├── tests
│    └── LoginTest.java
│
├── testdata
│    └── TestData.xlsx
│
└── runners
     └── TestNG.xml
```

🔷 2. Core Design Decisions (What you chose + My correction)

Area	Your Choice	Verdict
Framework Type	Hybrid	✅ Correct
Runner	TestNG + Maven	✅ Industry standard
Design Pattern	POM + Fluent	⚠️ Use Fluent **only for flows**
Execution	Parallel + ThreadLocal	✅ Correct
Test Data	Excel	⚠️ Acceptable but limited
Reporting	Extent + Log4j	✅ Correct

🔷 3. Key Components (No shortcuts here)

✅ DriverManager (ThreadLocal – MUST be correct)

```
public class DriverManager {

    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return driver.get();
    }

    public static void setDriver(WebDriver driverInstance) {
        driver.set(driverInstance);
    }

    public static void unload() {
        driver.remove();
    }
}
```

✅ BaseTest (Setup + Teardown)

```
@BeforeMethod
@Parameters("browser")
public void setup(String browser) {
    WebDriver driver = BrowserFactory.initDriver(browser);
    DriverManager.setDriver(driver);
}

@AfterMethod
public void tearDown() {
    DriverManager.getDriver().quit();
    DriverManager.unload();
}
```

✅ Page Layer (Clean – No business logic)

```
public class LoginPage {

    private By username = By.id("user");
    private By password = By.id("pass");
    private By loginBtn = By.id("login");

    public void enterUsername(String user) {
        DriverManager.getDriver().findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        DriverManager.getDriver().findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        DriverManager.getDriver().findElement(loginBtn).click();
    }
}
```

⚠️ Fluent Action Layer (Use smartly)

```
public class LoginActions {

    LoginPage loginPage = new LoginPage();

    public LoginActions login(String user, String pass) {
        loginPage.enterUsername(user);
        loginPage.enterPassword(pass);
        loginPage.clickLogin();
        return this;
    }
}
```

👉 Don’t over-chain everything. Keep flows readable.

✅ Excel Utility (Centralized – no duplication)

```
public class ExcelUtil {

    public static String getData(String sheetName, int row, int col) {
        // Apache POI logic
        return value;
    }
}
```

✅ Wait Utility (Explicit Wait – Mandatory)

```
public class WaitUtil {

    public static WebElement waitForElement(By locator) {
        WebDriverWait wait = new WebDriverWait(
            DriverManager.getDriver(), Duration.ofSeconds(10)
        );
        return
wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
}
```

✅ Extent Report + Listener

```
public class TestListener implements ITestListener {

    public void onTestFailure(ITestResult result) {
        ScreenshotUtil.capture();
        ExtentManager.getTest().fail(result.getThrowable());
    }
}
```

🔷 4. TestNG Parallel Execution

```
<suite name="Suite" parallel="tests" thread-count="3">
    <test name="Test1">
        <classes>
            <class name="tests.LoginTest"/>
```

```
        </classes>
    </test>
</suite>
```

🔷 5. Critical Best Practices (Most people fail here)

🚫 Common Mistakes You MUST avoid

- Using `Thread.sleep()` → unacceptable
- Writing WebDriver directly in test classes → bad design
- Mixing locators + business logic → poor maintainability
- Overusing Fluent Interface → debugging nightmare
- Reading Excel in test methods → performance issue

✅ What makes your framework "Industrial"

- Thread-safe execution
- Clear separation of layers
- Reusable utilities
- Proper reporting + logging
- Scalable structure
- CI/CD ready (next step for you)

🔷 Final Verdict (Straight Talk)

You made mostly solid choices, but:

- Excel choice limits long-term scalability
- Fluent Interface can backfire if overused

👉 Overall: **7.5/10 framework design**